Die Linux Shell

Dozent: Prof. Dr. Michael Eichberg
Kontakt: michael.eichberg@dhbw.de , Raum 149B
Version: 1.2.3

Eine kurze Wiederholung / Einführung.

Folien: [HTML] https://delors.github.io/lab-shell/folien.de.md.html
[PDF] https://delors.github.io/lab-shell/folien.de.md.html.pdf

Fehler melden: https://github.com/Delors/delors.github.io/issues

KI Verwendung: Bei der Erstellung der Unterlagen wurden KI Assistenten (insbesondere Claude aber ggf. auch ChatGPT, Ollama mit Gemma/LLama/Qwen oder OpenCode mit Kimi/Qwen/Deepseek...) unterstützend eingesetzt. Dies erfolgte insbesondere zur Unterstützung bei der Generierung von Grafiken (d. h. SVG Dateien), oder um sich Übersichtstabellen generieren zu lassen. Weiterhin wurde KI zur allgemeinen Qualitätssicherung eingesetzt. Inhalte, die ggf. von der KI vorgeschlagen wurden, wurden im Falle der Übernahme explizit validiert und angepasst.

1. Erste Schritte

Einloggen auf einem Server

SSH ist die Secure Shell und ein allg. etabliertes Werkzeug im Bereich der Administration von (Server-)Systemen, IoT Geräten etc.

Im professionellen Umfeld würde man zur Authentifizierung mittels SSH auf Zertifikate setzen. Für diese erste Übung verzichten wir darauf.

Ablauf

0. Starten Sie ggf. den CISCO Secure Client, wenn Sie auf interne Server der DHBW Mannheim zugreifen wollen. Dies ist ggf. auch aus dem WLAN der DHBW heraus notwendig!
1. Starten Sie eine Shell (z. B. Terminal oder iTerm auf MacOS bzw. Linux; cmd oder Powershell unter Windows.)
2. Verwenden Sie SSH, um sich auf dem Server einzuloggen. Sollte Ihr Accountname zum Beispiel `mueller` sein und der Server z. B. unter der IP `141.72.12.103` erreichbar sein, dann können Sie sich wie folgt auf dem Server einloggen *wenn passwort-basierte Authentifizierung möglich ist*[1]:

   ```
   ssh -l mueller 141.72.12.103
   ```

> **Hinweis**
>
> Im professionellen Umfeld ist es bei der Verwendung von SSH üblich, sich mittels Public-Key-Authentifizierung zu authentifizieren.

---

[1] Erhalten Sie eine Meldung, dass eine passwortbasierte Authentifizierung nicht erlaubt ist, dann müssen Sie sich an Ihren Administrator wenden.

Dateisysteme auf Unix-ähnlichen Betriebssystemen

- Die Verzeichnisstruktur in Unix-ähnlichen Systemen ist hierarchisch aufgebaut und beginnt bei der Wurzel (`/`).
- Alle Dateien und Verzeichnisse sind Unterknoten dieses Wurzelverzeichnisses.

Hinweis

In unixoiden Betriebssystemen trennt `/` auch Verzeichnisse.

Beispiel

`/Users/administrator/.zsh_history` ist der Pfad der Datei .zsh_history im Verzeichnis `administrator`, welches wiederum im Verzeichnis `Users` zu finden ist. `Users` ist ein direktes Kindverzeichnis des Wurzelverzeichnisses `/`.

- Im Regelfall sind Datei- und Verzeichnisnamen *Case-sensitive*.

Beispiel

Es ist möglich zwei Dateien `Test.txt` und `test.txt` gleichzeitig zu haben, die sich nur in der Groß-Kleinschreibung unterscheiden.

Ist ein Dateisystem nur *Case-preserving* (Standard unter Windows und MacOS), dann ist es zwar möglich z. B. eine Datei Test.txt anzulegen, aber nicht gleichzeitig auch eine Datei test.txt, die sich nur in der Kapitalisierung der Buchstaben unterscheidet.

- Geräte (wie zum Beispiel Festplatten, USB Sticks, Zufallszahlengeneratoren ...) können an *fast beliebiger Stelle* eingehängt werden.

Beispiel

Mittels *mount* kann man sich anzeigen lassen welche Geräte wo eingehängt sind:

```
/dev/vda2 on / type ext4 (rw,relatime)
/dev/vda1 on /boot/efi type vfat (rw,relatime,...)
```

- Systeminformationen sind auch direkt über spezielle Verzeichnisse verfügbar

Beispiel

Ausgabe von detaillierten Informationen über die CPU:

```
more /proc/cpuinfo
```

Ausgabe der Log-level Konfiguration des Kernels:

```
cat /proc/sys/kernel/printk
```

- Die Systemkonfiguration kann direkt über das Schreiben in spezialisierte Dateien erfolgen (meist sind Administratorrechte notwendig)

Beispiel

```
echo 7 > /proc/sys/kernel/printk
```

Setzt den Logging-level für das Kernel Logging auf den Debug-Level, in dem der Wert 7 in die Datei `printk` geschrieben wird.

`cat`, `more` oder auch `less` sind ganz generische Programme zum Lesen von jeder Art von Text-basierten Dateien.

Wichtige Verzeichnisse

Die wichtigsten Verzeichnisse - insbesondere unter Linux Betriebssystemen - sind:

Verzeichnis	Bedeutung / Inhalt
/	Wurzelverzeichnis – Ausgangspunkt aller Pfade
/bin	Wichtige Systemprogramme (z. B. `ls`, `cp`, `mv`)
/dev	Gerätedateien (z. B. Festplatten: `/dev/sda`, Terminals: `/dev/tty`)
/etc	Systemweite Konfigurationsdateien
/home	Persönliche Benutzerverzeichnisse (z. B. `/home/alex`)
/lib	Wichtige Systembibliotheken
/media	Einhängepunkte für Wechseldatenträger
/mnt	Temporärer Einhängepunkt für Administratoren
/root	Heimatverzeichnis des Superusers `root`
/sbin	Systemprogramme für den Administrator
/tmp	Temporäre Dateien
/usr	Sekundäres Hierarchiesystem für Benutzerprogramme
/var	Veränderliche Dateien (z. B. Logs, Mails, Spool-Dateien)
~	Alias für das Homedirectory des aktuellen Nutzer.

Grundlegendes

Verzeichnis Operationen

ls: Gibt Informationen über eine Datei/Verzeichnis aus. Ermöglicht es insbesondere den Inhalt eines Verzeichnisses aufzulisten. *ls* `-al` gibt weitergehende Informationen aus. Insbesondere alle versteckten Dateien und Verzeichnisse sowie die Größe der Dateien und die Rechte.

`cd`: (🇺🇸 *change directory*)
Wechsel des Verzeichnisses. Wechsel zu einem bestimmten Verzeichnis erfolgt durch Angabe des Names (`cd Test`). Wechsel zum Elternverzeichnis erfolgt über: `cd ..` (`..` steht immer für das Elternverzeichnis). Wechsel zum Wurzelverzeichnis erfolgt über: `cd /`

mkdir: Anlegen eines Verzeichnisses (*mkdir* `Test`)

rmdir: Löschen eines *leeren* Verzeichnisses (*rmdir* `Test`)

rm: (🇺🇸 *remove*) Löschen einer Datei (*rm* `Hello.txt`)

Allgemeines

- Wenn man in Linux/MacOS ein Stern benutzt, dann wird dies zu den Namen aller nicht-versteckten Dateien/Verzeichnisse im aktuellen Verzeichnis aufgelöst und diese werden an das Programm übergeben! Zum Beispiel löscht *rm* `*` alle Dateien im aktuellen Verzeichnis.
- Verfügbarer Speicher auf der Festplatte: *df* `-h`
- Benutzer Speicher (eines Verzeichnisses): *du* `-h` `-s` ~
- Inhalt einer Datei editieren ist (zum Beispiel) mit `pico` möglich. Alternative kann mit Hilfe von `echo` ein Text in eine Datei umgeleitet werden (`echo "Hello" > Hello.txt`).
- Inhalt einer Datei ansehen: `cat`, `less`, `more`, ...

Hilfe bekommen

In unixoiden Betriebssystemen gibt es für Kommandozeilentools Insbesondere die folgenden *eingebauten* Hilfemöglichkeiten:

- Hilfe des Programms (typischerweise `--help` oder einfach starten).
- **Manpages** (mittels *man* `<Name des Programms>`)
- Infopages (mittels `info <Name des Programms>`)

◆ Bemerkung

Manpages existieren für praktisch alle Kommandozeilenwerkzeuge.

Nicht bei allen Programmen erhält man über `--help` eine erste Hilfe.

Je nach Anwendung kann das Starten der Anwendung ohne Parameter dazu führen, dass das Programm auf Eingaben wartet (z. B. *cat*).

Kommandozeilenprogrammen beenden

<ctrl>+c: Programm abbrechen (ggf. unkontrolliert)

<ctrl>+d: Eingabe abschließen bzw. Ende der Eingabe

kill <PID>**:** Mittels des Befehls *kill* lässt sich das Programm auch beenden. Erfordert ggf. dass man einen zweiten Terminal startet um die PID zu ermitteln (zum Beispiel mittels *top* oder *ps*):

✎ Übung - mit einem Server verbinden

1.1. Unix - Erste Schritte

1. Loggen Sie sich auf dem Server unter Verwendung der passenden Zugangsdaten ein.
2. Ändern Sie Ihr Passwort mit Hilfe von `passwd`.

✎ Übung - mit einem Server über Keys verbinden[2]

1.2. Unix - Einloggen mittels SSHKey

Im professionellen Umfeld ist die Authentifizierung über SSH-Keys üblich. SSH-Keys sind eine sicherere Alternative zu Passwörtern, da keine Passwörter übertragen werden.

1. Erstellen Sie **lokal** - auf Ihrem Clientrechner - einen SSH Key-Paar mittels `ssh-keygen`. Nutzen Sie den `ed25519` Algorithmus.

   Sollten Sie bereits ein SSH Key-Paar haben, dann können Sie diesen verwenden. Ansonsten erstellen Sie das Schlüsselpaar im lokalen Verzeichnis.

2. Kopieren Sie den **öffentlichen Schlüssel** auf den Server mittels `ssh-copy-id`.

   Der öffentliche Schlüssel ist in der `.pub` Datei. Diese Datei ist das *identity file*, dass der Server benötigt/verwendet.

3. Testen Sie ob Sie sich beim Server anmelden können ohne Passwort!

---

[2] Nutzen Sie Google/LLMs nur, wenn die Manpages und die Hilfe der Programme Ihnen nicht weiterhelfen.

✎ Übung - Starten eines kleinen WebServers

1.3. Unix - Grundlegende Operationen

1. Erzeugen Sie ein Verzeichnis mit Ihrem Nachnamen (z.B. "eichberg")
2. Legen Sie in dem Verzeichnis eine rudimentäre `index.html` Datei an.

   (Sie können zum Beispiel `pico`, `vim` oder `echo` verwenden!)
3. Starten Sie einen Webserver mit dem gerade angelegten Verzeichnis als Wurzel. Wählen Sie eine zufälligen Port XXXX > 1024 aus.

   ### IPv4

   Zum Beispiel können Sie den bereits installierten Webserver mittels `http-server` wie folgt starten. Im Folgenden ist der Port `XXXX` und das Wurzelverzeichnis für Ihre Webseiten "`eichberg`": `http-server -p XXXX eichberg`.

   ### IPv6

   Zum Beispiel können Sie den bereits installierten Webserver mittels `http-server` wie folgt starten. Im Folgenden ist der Port `XXXX` und das Wurzelverzeichnis für Ihre Webseiten "`eichberg`": `http-server -a ::0 -p XXXX eichberg`.

   ### Auf Server zugreifen:

   Entweder wie gewohnt: `http://<URL>:<PORT>/index.html` oder falls der Server nur IPv6 spricht und keinen Hostnamen hat: `http://[<IPv6>]:<PORT>/index.html` (z.B. `http://[2001:7c0:1b20:c913:1::3d8]:3333`) - die eckigen Klammern sind obligatorisch!
4. Wechseln Sie in Ihren Browser und öffnen Sie die entsprechende Webseite.

   (Denken Sie daran, dass dieser Server nur http und nicht https unterstützt.)
5. Beenden Sie den WebServers
6. Löschen Sie das gerade angelegte Verzeichnis.

✎ Übung - Starten des Servers im Hintergrund

1.4. nohup

1. Starten Sie den Webserver im Hintergrund und leiten Sie die Ausgaben in eine Log-Datei um. Nutzen Sie `nohup` als Tool.

   Beispiel

   IPv4: `nohup http-server -p 8888 eichberg > web.log 2>&1 &`

   IPv6: `nohup http-server -a ::0 -p 8888 eichberg > web.log 2>&1 &`

2. Überprüfen Sie, ob der Server im Hintergrund läuft. D. h. loggen sie sich aus und rufen Sie danach wieder die Webseite auf.
3. Loggen Sie sich wieder ein und beenden Sie den Server mittels `kill` Befehl. Die PID können Sie mittels `ps -aux | grep http-server` herausfinden.

✎ Übung - Installation von Software als normaler Nutzer

1.5. NVM installieren

Installieren Sie nvm (Node Version Manager), um im zweiten Schritt eine aktuelle Version von node.js zu installieren.

https://github.com/nvm-sh/nvm?tab=readme-ov-file#installing-and-updating

✎ Übung - Daten transferieren

1.6. Secure Copy (*scp*) nutzen

Nutzen Sie das Programm *scp*, um eine HTML Datei von Ihrem lokalen Rechner auf den Zielserver zu kopieren.

1. Nutzen Sie die Hilfe zu *scp*, um herauszufinden wie Sie eine Datei auf den Server in das richtige Verzeichnis kopieren können.
2. Öffnen Sie Ihren Browser, um zu prüfen, dass Sie auf die kopierte Webseite zugreifen können.

2. (Text-)Dateien und Ihre Bearbeitung

Die grundlegenden Datenströme

Jedes Programm hat immer Zugriff auf die drei Standarddatenströme:

- `stdin` - Eingabedatenstrom.
- `stdout` - Ausgabedatenstrom für „normale“ Nachrichten.
- `stderr` - Ausgabedatenstrom für Fehlernachrichten.

Diese Datenströme sind (auch nur) Dateien, mit einem festgelegten Dateideskriptor (🇺🇸 *file descriptor*):

0:	`stdin`
1:	`stdout`
2:	`stderr`

Umleitung von `stdin` in Dateien

> leitet die Ausgabe eines Programmes/Kommandos in eine Datei um; löscht bzw. legt die Zieldatei bei Bedarf an.

>> leitet die Ausgabe eines Programmes/Kommandos in eine Datei um; hängt die Ausgabe an das Ende einer bestehenden Datei an bzw. legt die Zieldatei bei Bedarf an.

Beispiel

Erzeugen einer Datei `tmp/0s.txt`, die 1024-mal den Wert 0 in Base64 Kodierung enthält.

```
dd if=/dev/zero bs=1 count=1024 | base64 \ # 1024 * "0" in base64
>                                       \ # Umleitung der Ausgabe
/tmp/0s.txt                               # in /tmp/0s.txt
```

Bemerkung

Der \ am Zeilenende wird dazu verwendet, lange Shellkommandos über mehrere Zeilen schreiben zu können; # leitet ein "End-of-line Comment" ein.

Umleiten von bestimmten Ausgaben an Dateien

Beim Umleiten von Ausgaben an eine Datei, kann der Dateideskriptor angegeben werden: `[FD]><Ziel>`

`2>` leitet z. B. die Fehlerausgaben (🇺🇸 *stderr*) eines Programmes/Kommandos in eine Datei um; löscht bzw. legt die Zieldatei bei Bedarf an.

Beispiel

Finden von bestimmten Dateien; aber Fehlerausgaben während des Suchprozesses ignorieren.

```
find / -iname "*txt*" -type f \
2>/dev/null           # Umleitung aller Fehler nach /dev/null
```

Grundlegende Prinzipien: Lesen aus einer Datei

< liest den Inhalt einer Datei und leitet diesen an das Programm/Kommando weiter; d. h. stellt den Inhalt über stdin zur Verfügung.

Beispiel

Finden aller Städte, die mit "B" beginnen.[3]

```
grep B \             # filtert alle Zeilen, die ein "B" enthalten
   < Big\ Cities.txt # der Inhalt von Big Cities.txt wird über stdin
                     # zur Verfügung gestellt
```

[3] In diesem Fall könnte die Datei (`Big Cities.txt`) auch direkt als Parameter an `grep` übergeben werden. In anderen Fällen ist dies aber nicht möglich.

Linux Shell - Grundlegendes Design-Pattern: **Pipes and Filters**

- Grundlegendes Konzept bzw. Entwurfsmuster (*Design-Pattern*) in Unix-basierten Betriebssystemen.
- Ermöglicht die effiziente Verkettung von Befehlen. Die „Pipes-and-Filter“ Architektur erlaubt es komplexe Verarbeitungsoperationen mit Hilfe der Kombination von einfachen Befehlen durchzuführen.

`|`: Verbindet den Ausgabestrom (`stdout`) des vorhergehenden Befehls mit dem Eingabestrom (`stdin`) des nachfolgenden Befehls.

Beispiel

Konvertierung des Wortes `Test` in Base64 Kodierung.

```
echo -n "Test" \
|              \ # Weiterleitung der Ausgabe von echo an base64.
base64
```

„Filter“: Kommandos/Programme, die von `stdin` lesen und nach `stdout` schreiben.

Wichtige Linux Kommandozeilenwerkzeuge für die Verarbeitung von Passwortkandidaten

cat:	Dateien verketten.
sed:	Strom Editor.
grep:	Mustersuche auf Dateien.
tr:	Ersetzung und Löschung von Zeichen.
uniq:	Filtert wiederholte aufeinanderfolgende Zeilen in einer Datei.
sort:	Sortiert Dateien.
echo:	Schreibt Argumente auf *Standard Out* (`stdout`).
wc:	Zählt die Zeichen, Wörter, Zeilen einer Datei.
comm:	Vergleicht sortierte Listen und filtert entsprechend.
find:	Auswertung eines Ausdrucks für jede Datei während eines rekursiven Abstiegs über den Verzeichnisbaum.
awk:	Muster-orientierte Verarbeitung der Zeilen einer Eingabedatei.
base64:	(De-)Kodierung von Daten in Base64 Kodierung.
rev:	Dreht die Reihenfolge der Zeichen einer Zeile um.
head:	Zeigt die ersten (`-n`) Zeilen einer Datei an.
tail:	Zeigt die letzten (`-n`) Zeilen einer Datei an. (`-f` folgt der Datei, d. h. wartet auf weitere Daten, die der Datei hinzugefügt werden.)

Anwendungsfälle

Typischerweise werden diese Werkzeuge bei der Verarbeitung von Leaks/Aufbereitung von Wörterbüchern im Vorfeld gebraucht - vor dem eigentlichen Versuch das Passwort wiederherzustellen.

echo

- Universell eingesetzt, um Inhalte in Dateien zu schreiben bzw. anzuhängen.
- `-n` um das automatische Anhängen von Zeilenumbrüchen zu unterdrücken. (Besonders dann wichtig, wenn man Hashes für Testdaten generieren will.)
- Entweder ein explizites Programm oder ein in die Shell eingebautes Kommando.

Anwendungsfall: Programmatisch Daten nach `stdout` schreiben.

```
$ echo -n "TestPasswort"
  | shasum -a 256
  | sed -E 's/  -$//'
2214db3d6fca761041242b9fc41fdcca
f0b2c7f556b80c0a91cfe6994437d807
```

> ◎ Bemerkung
>
> Der hier zu sehende Befehl `shasum -a 256` ist unter einigen Linuxdistributionen einfach `sha256sum`.

cat

- Liest alle Dateien sequentiell ein und schreibt diese auf `Standard Out` (stdout).
- "`-`" repräsentiert `Standard In` (stdin); dies ermöglicht die Verwendung von cat mitten in einer Verarbeitungskette.
- Liest (ggf.) von `stdin` bis zur EOF 🇺🇸 *End-of-File* Markierung.

  (Das Einlesen von der Kommandozeile kann mit `CTR+D` beendet werden.)

Anwendungsfall: Mehrere Teilwörterbücher sollen zusammengefügt werden.

Inhalt von Test1.txt: `Test1`

Inhalt von Test2.txt: `Test2`

```
$ echo "Test3" | cat Test1.txt Test2.txt -
Test1
Test2
Test3
```

tr

- Kopiert die Eingabe von `stdin` nach `stdout` und führt dabei Substitutionen und Löschungen durch.

Anwendungsfall: bestimmte Buchstaben - zum Beispiel Sonderzeichen - sollen gelöscht werden.

```
$ echo -n 'ab.cd_12!' | tr -dc '[:alnum:]'  # -dc = delete complement
abcd12
```

Anwendungsfall: Groß- in Kleinbuchstaben verwandeln.

```
$ echo -n 'STARK' | tr '[:upper:]' '[:lower:]'
stark
```

uniq

- vergleicht nebeneinanderliegende Zeilen und schreibt jede einzigartige Zeile einmal nach `stdout`. Nicht-nebeneinanderliegende Wiederholungen werden nicht erkannt.
- `-c` erlaubt es die Anzahl der Wiederholungen zu zählen.

Anwendungsfall: Wir möchten eine alphabetisch sortierte Liste nach der Häufigkeit des Vorkommens eines Wortes sortieren.

Mittels `uniq` kann die Häufigkeit gezählt werden.

Die Sortierung - zum Beispiel angefangen mit den am häufigsten vorkommenden Einträgen - kann danach im Nachgang erfolgen.

```
$ echo "Test\nTest\nSchlaraffenland\nTest" | uniq -c
2 Test
1 Schlaraffenland
1 Test
```

awk

- Muster-orientierte Verarbeitung der Zeilen einer Eingabedatei.
- Jede Zeile wird segmentiert (Standardmäßig basierend auf Leerzeichen), die einzelnen Segmente werden mit `$1`, `$2`, … bezeichnet. `$0` steht für die ganze Zeile.
- Die Verarbeitung erfolgt durch Muster-Handlungsanweisungen der Form:

  ```
  pattern { action }
  ```

  ist das Muster (🇺🇸 *pattern*) leer, dann wird die Zeile immer verarbeitet; ist keine Handlungsanweisung (🇺🇸 *action*) angegeben, dann wird die Zeile ausgegeben.

Anwendungsfall: Die Einträge einer Datei sollen nach länge sortiert werden. In diesem Fall, kann mit Hilfe von awk jede Zeile mit der Länge ausgegeben werden. Danach kann die Liste entsprechend sortiert werden.

```
$ echo "Test\nSchlaraffenland" |  awk '{print length " " $1}'
4 Test
15 Schlaraffenland
```

sort

- Sortiert eine Liste gemäß der entsprechenden Felder.
- `-r` sortiert in absteigender Reihenfolge.
- `-n` der Wert des ersten Feldes wird als numerischer Wert interpretiert.
- `-k` spezifiziert das Feld, nach dem sortiert werden soll. (z. B. -k 3)
- `-t` spezifiziert das Trennzeichen, das die Felder trennt. (z. B. -t ',')

Anwendungsfall: Sortiere eine Liste nach Häufigkeit des Vorkommens eines Wortes.

```
$ echo "abc\nxyz\nuvw\nxyz" \
  | sort \                  # alphabetische Sortierung
  | uniq -c \               # zähle Häuigkeit des Vorkommens einer Zeile
  | sort -nr \              # absteigende Sortierung
  | sed -E 's/ *[0-9]+ *//' # entferne den Zähler
xyz
uvw
abc
```

Komplexes Beispiel

Sortierung einer Liste von Worten in absteigender Reihenfolge bzgl. (1) der Häufigkeit und (2) Länge.

```
$ printf '%s' "abc\nuvw\nxyz\nlmnop\nxyz\nuvw" \
       "\nlmnop\nlmnop\nxyz\ncd\ncd\ncd" \
  | awk '{print length " " $1}'
  | sort
  | uniq -c
  | sort -nr -k 1 -k 2
3 5 lmnop
3 3 xyz
3 2 cd
2 3 uvw
1 3 abc
```

Sortierung einer Liste von Worten in absteigender Reihenfolge bzgl. (1) der Häufigkeit und (2) aufsteigend bzgl. der Länge.

```
$ echo "abc\n" "uvw\n" "xyz\n" "lmnop\n" "xyz\n" "uvw\n" \
       "lmnop\n" "lmnop\n" "xyz\n" "cd\n" "cd\n" "cd" \
  | awk '{print length " " $1}' \
  | sort | uniq -c \
  | sort  -k1nr -k2n
3 3 cd
3 4 xyz
3 6 lmnop
2 4 uvw
1 3 abc
```

base64

Base64 kodierte Werte bestehen nur noch aus gültigen ASCII Zeichen und können als "Text" gespeichert/übermittelt werden kann.

> ◎ Bemerkung
>
> Je nach Betriebssystem/Konfiguration ist der Befehl unter Umständen etwas anders benannt. Grundlegend gibt es den Befehl auf allen Unixoiden.

Anwendungsfall: In vielen Fällen können gehashte Passworte nicht roh (d. h. als Binärdaten) gespeichert werden, sondern müssen Base64 (oder vergleichbar) kodiert werden.

```
# Codierung
$ echo "Dies_ist_ein_test" | base64
RGllc19pc3RfZWluX3Rlc3QK
$ echo 'Dies_ist_ein_test!' | base64
RGllc19pc3RfZWluX3Rlc3QhCg==

# Dekodierung
$ echo REhCVyBNYW5uaGVpbQ== | base64 --decode
DHBW Mannheim
```

grep

- Selektiert Zeilen, die einem gegebenen Muster entsprechen.
- `-o` gibt nur den Teil einer Zeile aus, der dem Muster entspricht.
- `-v` selektiert Zeilen für die kein Teil der Zeile dem Muster entspricht.
- `-E` erlaubt die Spezifikation von Mustern mit Hilfe von regulären Ausdrücken.
- `-i` ignoriert Groß-/Kleinschreibung (in Verbindung mit -E mgl. verwirrend).
- `-P` Perl kompatible Ausdrücke

Anwendungsfall: Alle Textfragmente in einem Leak finden, um danach mit Regeln neue Passwort-kandidaten zu bilden.

```
$ echo "Test123\nmichael@dhbw.de\n345Test@dhbw.de\nEnde__" \
  | grep -Eo "[a-zA-Z]{3,}" | sort -u
Ende
Test
dhbw
michael
```

sed - Stromeditor

- modifiziert die Eingabe gemäß der spezifizierten Kommandos in der angegebenen Reihenfolge.
- `-E` zur Verwendung moderner regulärer Ausdrücke
- Standardform: `Funktion[Agrumente]`
- Substitutionen: `s/Regulärer Ausdruck/Ersatz/[Kennzeichen]`; das Kennzeichen "`g`" z. B. bewirkt, dass jedes Vorkommen ersetzt wird; sonst nur das erste Vorkommen.

Anwendungsfall: Löschen des ersten Sonderzeichens in einer Zeile.

```
$ echo 'ab_cd!_ef?' | sed -E  's/[^a-zA-Z0-9]//'
abcd!_ef?
```

Anwendungsfall: Analyse der Struktur eines Leaks durch das Abbilden **aller** Buchstaben auf die Repräsentanten: `l`(lower) `u`(upper) `d`(digits) `s`(special).

```
$ echo 'aB_c1d!_ef?' |
sed -E -e's/[a-z]/l/g' -e's/[A-Z]/u/g' -e's/[0-9]/d/g' -e 's/[^lud]/s/g'
lusldlssls
```

Hinweis

`sed` auf dem Mac (BSD) und `sed` unter Linux (GNU) unterscheiden sich teilweise deutlich.

find

- durchläuft den Dateibaum ab einer angegebenen Stelle und evaluiert dabei Ausdrücke.
- `-iname` Testet ob der Verzeichniseintrag - unabhängig von der Groß- und Kleinschreibung - dem gegebenen Muster entspricht.
- `-exec ... {} ... \;` ermöglicht es für jede gefilterte Datei `{}` einen Befehl auszuführen.

Anwendungsfall: Feststellen wie lange die Hashes sind.

```
$ find . -iname "*hash*" -exec wc -c {} \;
33 ./saltedmd5/hash.md5
38 ./saltedmd5/saltedhash.md5
129 ./scenario5/hash.sha125
65 ./scenario6/hash.sha256
65 ./scenario7/hash.sha256
65 ./scenario9/hash.sha256
```

Software nachinstallieren

- Auf allen Linux und BSD Distributionen können Softwarepakete durch den Paketmanager des Betriebssystems nachinstalliert werden, z. B.:
  - `apt` (Debian, Ubuntu, Kali Linux, ...)
  - `yum` (RedHat, CentOS, ...)
  - `pacman` (Arch Linux, ...)
  - `brew` oder `macports` (MacOS) [4]

Anwendungsfall: Installieren von `ent` (ein Programm, das die Entropie von Dateien berechnet):

```
1 sudo apt install ent
```

◆ Bemerkung

Die meiste Software (z. B. Laufzeitumgebungen wie NodeJs) lassen sich auch als normaler Nutzer für die eigene Nutzung installieren.

---

[4] Beide sind in diesem Fall nicht Teil des Betriebssystems, sondern müssen erst nachinstalliert werden, bevor damit weitere Software nachinstalliert werden kann.

Shellprogrammierung

- Jede Shell (insbesondere: `zsh` (auf Mac und Kali Linux) und `bash` (Debian, Ubuntu, ...)) erlaubt es prozedurale Programme zu schreiben.

Anwendungsfall: Berechnung der Entropie für jede Datei in einer Liste.

```
#!/usr/bin/zsh                  # Shebang zur Spezifikation der Shell
IFS=$'\n'                       # IFS = Internal Field Separator
                                # (Nur Zeilenumbrüche sind Trennzeichen)
rm Files.list.assessed          # Lösche die Ausgabedatei
for i in $(cat Files.list); do  # Iteriere über die Zeilen in Files.list
    echo "Processing: ""$i"
    ent -t "$i" | \             # Berechne die Entropie
    grep -E "^1" | \            # Selektiere die Zeile mit der Entropie
    tr -d '\n' | \              # Lösche den Zeilenumbruch
    cat - <(echo ","""$i") \    # Füge den Dateinamen hinzu
        >> Files.list.assessed ; # Schreibe das Ergebnis
done;
```

✎ Fingerübungen

Voraussetzung

Starten Sie z. B. Kali Linux (oder eine entsprechende VM), loggen Sie sich ein und starten Sie ein Terminal.

2.1. Dateien finden

Finden Sie die Datei, die die Standardpassworte von Postgres Datenbanken enthält (der Dateiname enthält sowohl `postgres` als auch `pass`).

2.2. MD5 Hash berechnen

Konkatenieren sie die Zeichenkette „MySalt“ (ohne Zeilenumbruch!) mit dem Inhalt von rockyou.txt (als Ganzes) und berechnen Sie davon den md5 Hash. Verwenden Sie keine expliziten Zwischenergebnisse.

2.3. Base64

Erzeugen Sie für eine Datei (z. B. `/usr/bin/wc`) einen MD5 hash und stellen Sie diesen der Datei selber voran bevor sie alles nach Base64 konvertieren.